

Kalman Filter for Dynamic Pairs Trading

`kalman_filter.py` — Mathematical Notes & Documentation

Bayesian State Estimation, Predict-Update Recursion, MLE Parameter Tuning

Project Dark-Wing | July 26, 2026

Abstract. In pairs trading, OLS gives a *constant* hedge ratio $\hat{\beta}_{\text{OLS}}$ that ignores the empirical fact that market relationships evolve over time. The Kalman filter treats β_t as a *hidden state* that evolves as a random walk and is observed noisily through the co-movement of two asset prices. At each timestep it *predicts* the next β , then *corrects* that prediction with the new price data via the famous *Kalman Gain*. This document derives every formula in `kalman_filter.py` from first principles using the state-space model notation from your Notion notes (S_t, T_t, Z_t, Ω), includes a full worked numerical example, and provides TikZ flowcharts of the predict/update cycle.

Why Dynamic β ?

Static OLS computes a single $\hat{\beta}$ by minimising $\sum (Y_t - \alpha - \beta X_t)^2$. If the relationship between two stocks shifts — due to sector rotation, regulatory changes, or earnings divergence — $\hat{\beta}_{\text{OLS}}$ becomes stale.

Problem with Static β

OLS: $\hat{\beta} = \arg \min_{\beta} \sum_t (Y_t - \beta X_t)^2$

Assumes β is *constant over all time*. If β_t changes, OLS produces biased estimates and the spread $S_t = Y_t - \hat{\beta} X_t$ is non-stationary.

Kalman solution. Model β_t as a hidden state that evolves slowly over time (random walk). Each new (Y_t, X_t) pair is used to update our belief about β_t .

Bayesian Foundation

The Kalman filter is **recursive Bayesian estimation for a Gaussian linear system**. From your Notion notes:

$$\underbrace{P(\beta_t)}_{\text{prior}} \xrightarrow{\text{observe } Y_t} \underbrace{P(\beta_t | Y_t)}_{\text{posterior}}$$

Bayes' Rule for Kalman

$$P(\beta | Y) = \frac{P(Y | \beta) P(\beta)}{P(Y)} \quad (1)$$

For Gaussian distributions this has a closed-form solution: the prior belief $\beta_{t|t-1}$ is updated to $\beta_{t|t}$ using the innovation ε_t .

State-Space Model

General-Form Notation (from Notion Notes)

General State-Space Model

$$\text{State Eq.: } S_t = T_t S_{t-1} + R_t \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q_t) \quad (2)$$

$$\text{Obs. Eq.: } Y_t = Z_t S_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, H_t) \quad (3)$$

Symbol	Meaning	Pairs Role
S_t	State vector	Hedge ratio β_t
T_t	State transition	$T = 1$ (random walk)
Z_t	Observation matrix	X_t (indep. price)
Y_t	Observation	Y_t (dep. price)
Q_t	Process noise var.	How fast β changes
H_t	Meas. noise var.	R = spread noise
$\Omega_{s(t t-1)}$	State covariance	$P_{t t-1}$

Pairs Trading Specialisation

For our univariate pairs model, all matrices collapse to scalars:

Pairs-Trading State Space

$$\underbrace{\beta_t}_{\text{state}} = \underbrace{1}_T \cdot \beta_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q) \quad (4)$$

$$\underbrace{Y_t}_{\text{obs}} = \underbrace{X_t}_{Z_t} \cdot \beta_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R) \quad (5)$$

Two noise parameters control the filter:

- Q : process noise — how fast β drifts. Small Q = slow, stable β .
- R : measurement noise — how noisy the observed spread is.

The Predict Step

State Prediction

Before any new data arrives at time t , we project our best estimate of β forward:

Formula 1 — State Prediction

$$\hat{\beta}_{t|t-1} = T \cdot \hat{\beta}_{t-1|t-1} = \hat{\beta}_{t-1|t-1} \quad (6)$$

Since $T = 1$ (random walk), the best prediction for tomorrow's β is today's β .

Covariance Prediction

Our uncertainty grows because β_t may have drifted:

Formula 2 — Covariance Prediction

$$P_{t|t-1} = T \cdot P_{t-1|t-1} \cdot T' + Q = P_{t-1|t-1} + Q \quad (7)$$

From Notion notation: $\Omega_{s(t|t-1)} = T_t \Omega_{s(t-1|t-1)} T_t' + R_t Q_t R_t'$
Simplified (scalar, $R_t = 1$): $P_{t|t-1} = P_{t-1|t-1} + Q$.

Intuition. Every period that passes without an update, our uncertainty P grows by Q . A larger Q means we expect β to jump more, so we become uncertain faster.

The Update (Correction) Step

When price data (Y_t, X_t) arrives, we correct our prediction.

Innovation (Prediction Error)**Formula 3 — Innovation ε_t**

$$\varepsilon_t = Y_t - Z_t \hat{\beta}_{t|t-1} = Y_t - X_t \hat{\beta}_{t|t-1} \quad (8)$$

This is the *surprise*: how much the actual Y_t differed from what we expected, given our predicted β .

Innovation Covariance (Observation Uncertainty)**Formula 4 — Innovation Variance F_t**

$$F_t = Z_t P_{t|t-1} Z_t' + R = X_t^2 P_{t|t-1} + R \quad (9)$$

From Notion: $\Omega_{y(t|t-1)} = Z_t \Omega_{s(t|t-1)} Z_t' + \Omega_{\varepsilon_t}$
Simplified: $F_t = X_t^2 P_{t|t-1} + R$.

Financial meaning. F_t is the total uncertainty in our price prediction. It has two components:

- $X_t^2 P_{t|t-1}$: uncertainty from not knowing β_t exactly.
- R : pure measurement noise (bid-ask, data quality).

Kalman Gain G_t

The Kalman Gain decides *how much* to trust the new data vs. our prediction:

Formula 5 — Kalman Gain G_t

$$G_t = \Omega_{s(t|t-1)} Z_t' [\Omega_{y(t|t-1)}]^{-1} = \frac{P_{t|t-1} \cdot X_t}{F_t} \quad (10)$$

- If $F_t \uparrow$ (noisy observation): $G_t \downarrow \rightarrow$ trust prediction more.
- If $P_{t|t-1} \uparrow$ (uncertain prediction): $G_t \uparrow \rightarrow$ trust data more.

State and Covariance Update**Formula 6 — State Update**

$$\hat{\beta}_{t|t} = \hat{\beta}_{t|t-1} + G_t \varepsilon_t \quad (11)$$

The updated estimate is the predicted estimate plus a fraction G_t of the prediction error ε_t .

Formula 7 — Covariance Update

$$P_{t|t} = (I - G_t Z_t) P_{t|t-1} = (1 - G_t X_t) P_{t|t-1} \quad (12)$$

$$= \Omega_{s(t|t-1)} [I - G_t^T Z_t] \quad (\text{Notion form}) \quad (13)$$

After the update, uncertainty is *reduced* (P decreases) because we've just seen new data. This is the key feature of the filter.

Spread and Log-Likelihood**Formula 8 — Spread and Log-Lik**

$$S_t = Y_t - \hat{\beta}_{t|t} \cdot X_t \quad (14)$$

$$\ell_t = -\frac{1}{2} \left[\ln(2\pi F_t) + \frac{\varepsilon_t^2}{F_t} \right] \quad (15)$$

S_t uses the *updated* $\hat{\beta}_{t|t}$ (posterior). ℓ_t is the log-likelihood contribution used for Q/R optimisation.

Worked Numerical Example**Setup**

$Y = [100, 105, 102]$, $X = [50, 52, 51]$
 $\hat{\beta}_0 = 2.0$, $P_0 = 1000$, $Q = 0.001$, $R = 0.01$

Timestep $t = 1$: ($Y_1 = 100$, $X_1 = 50$)

$$\hat{\beta}_{1|0} = 2.0$$

$$P_{1|0} = 1000 + 0.001 = 1000.001$$

$$\varepsilon_1 = 100 - 2.0 \times 50 = 0.0$$

$$F_1 = 50^2 \times 1000.001 + 0.01 = 2,500,002.51$$

$$G_1 = \frac{1000.001 \times 50}{2,500,002.51} \approx 0.02$$

$$\hat{\beta}_{1|1} = 2.0 + 0.02 \times 0.0 = 2.0$$

$$P_{1|1} = (1 - 0.02 \times 50) \times 1000.001 \approx 0.01$$

Timestep $t = 2$: ($Y_2 = 105$, $X_2 = 52$)

$$\hat{\beta}_{2|1} = 2.0$$

$$P_{2|1} = 0.01 + 0.001 = 0.011$$

$$\varepsilon_2 = 105 - 2.0 \times 52 = 1.0$$

$$F_2 = 52^2 \times 0.011 + 0.01 = 29.72$$

$$G_2 = \frac{0.011 \times 52}{29.72} \approx 0.0192$$

$$\hat{\beta}_{2|2} = 2.0 + 0.0192 \times 1.0 = 2.019$$

$$P_{2|2} = (1 - 0.0192 \times 52) \times 0.011 \approx 0.0000388$$

Interpretation

After $t = 1$: with a perfect prediction ($\varepsilon_1 = 0$), β stays at 2.0 but P collapses dramatically ($1000 \rightarrow 0.01$) — the filter now trusts the data strongly.

After $t = 2$: Y_2 was higher than predicted ($\varepsilon_2 = +1$), so β_t increases from 2.0 $\rightarrow$ 2.019. The filter has learned the relationship is slightly stronger than initially thought.

Financial Interpretation Table

Notion	Math	Finance	Code
$S_{t t}$	$\hat{\beta}_{t t}$	Current hedge ratio	<code>beta</code>
$\Omega_{s(t t)}$	$P_{t t}$	Uncertainty in β	<code>P</code>
$S_{t t-1}$	$\hat{\beta}_{t t-1}$	Yesterday's β	<code>beta_pred</code>
$\Omega_{s(t t-1)}$	$P_{t t-1}$	Growing uncertainty	<code>P_pred</code>
ε_t	$Y_t - X_t \hat{\beta}_{t t-1}$	Spread error	<code>innovation</code>
$\Omega_{y(t t-1)}$	$F_t = X_t^2 P + R$	Obs. uncertainty	<code>F</code>
G_t	PX/F	Trust in new data	<code>kalman_gain</code>
Q	process noise var.	β evolution speed	<code>Q</code>
R/H	meas. noise var.	Market noise	<code>R</code>

Q/R Sensitivity and Tuning

The two key hyperparameters control the filter's behaviour:

Q/R Intuition

Large Q / Small R (trust data):

- β_t changes rapidly (agile filter)
- Kalman Gain G_t stays high
- Spread S_t is volatile; many signals

Small Q / Large R (trust model):

- β_t barely moves (conservative filter)
- G_t is small; ignores noisy data
- Closer to static OLS $\hat{\beta}$

Optimal: set via MLE optimising $\sum_t \ell_t$ over (Q, R) .

MLE Optimisation

Formula 9 — MLE for Q, R

$$(Q^*, R^*) = \arg \max_{Q, R > 0} \sum_{t=1}^T \ell_t(Q, R) \quad (16)$$

$$= \arg \min_{Q, R > 0} \sum_{t=1}^T \left[\ln F_t + \frac{\varepsilon_t^2}{F_t} \right] \quad (17)$$

Solved via L-BFGS-B (scipy.optimize.minimize) with bounds. Note: F_t and ε_t both depend on Q and R through the recursive filter equations.

Code Flowchart — KalmanFilterPairs

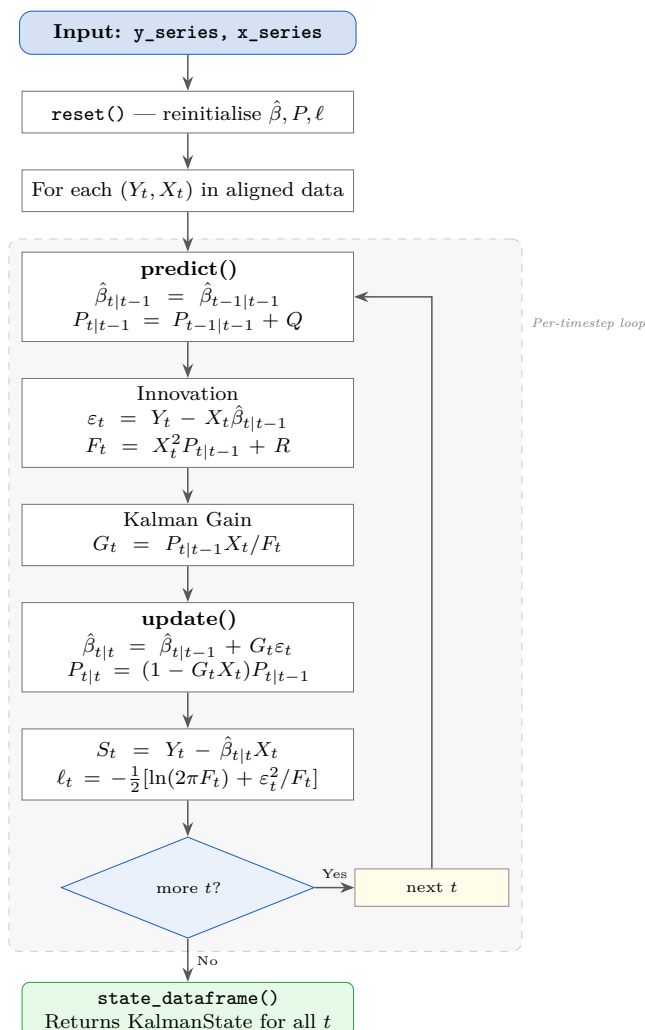


Figure 1: Execution flowchart for `KalmanFilterPairs.filter()`. The loop (shaded) runs predict-update at every timestep.

Python Implementation

Constructor

```
class KalmanFilterPairs:
    def __init__(self, Q=1e-5, R=1e-3,
                  beta_init=1.0, P_init=1000.0):
        self.Q = Q; self.R = R
        self.beta_init = beta_init
        self.P_init = P_init
        self._beta = beta_init
        self._P = P_init
        self.states = []
        self._log_lik_sum = 0.0
```

predict() and update()

```
def predict(self):
    beta_pred = self._beta          # T = 1
    P_pred = self._P + self.Q      # grow by Q
    return beta_pred, P_pred

def update(self, y, x, t=0):
    b_pred, P_pred = self.predict()
    innov = y - b_pred * x          # Formula 3
    F = x**2 * P_pred + self.R      # Formula 4
    G = P_pred * x / F              # Formula 5
    self._beta = b_pred + G*innov    # Formula 6
    self._P = (1-G*x)*P_pred        # Formula 7
    spread = y - self._beta * x      # Formula 8a
    log_lik = -0.5*(np.log(2*np.pi*F)
                  + innov**2/F)      # Formula 8b
    self._log_lik_sum += log_lik
    # ... store KalmanState and return
```

optimize_QR() via MLE

```
def optimize_QR(self, y_series, x_series,
                Q_bounds=(1e-8, 1e-1),
                R_bounds=(1e-6, 1.0)):
    def neg_loglik(params):
        Q, R = params
        tmp = KalmanFilterPairs(Q=Q, R=R)
        for yi, xi in zip(y_arr, x_arr):
            tmp.update(yi, xi)
        return -tmp.log_likelihood()
    res = minimize(neg_loglik, [self.Q, self.R],
                  method="L-BFGS-B",
                  bounds=[Q_bounds, R_bounds])
    if res.success:
        self.Q, self.R = res.x
```

Numerical Stability & The Joseph Form

In floating-point arithmetic, the standard covariance update equation:

$$P_{t|t} = (1 - G_t X_t) P_{t|t-1} \quad (18)$$

can suffer from numerical issues. Specifically, if the measurement update is extremely precise, $P_{t|t}$ can become very close to zero. Under roundoff errors, $P_{t|t}$ can occasionally drift slightly negative, which violates the covariance matrix property of positive-semidefiniteness and leads to filter collapse.

To guarantee positive-definiteness, we can use the **Joseph Form** covariance update. By expanding the error covariance definition:

$$e_{t|t} = \beta_t - \hat{\beta}_{t|t}$$

and substituting the state update equation, we obtain:

The Joseph Form Covariance Update

$$P_{t|t} = (1 - G_t X_t) P_{t|t-1} (1 - G_t X_t) + G_t^2 R \quad (19)$$

Since this is a sum of two squared terms (weighted by $P_{t|t-1} > 0$ and $R > 0$), the resulting covariance $P_{t|t}$ is guaranteed to be strictly positive at every step, regardless of roundoff error.

This Joseph form is implemented in the covariance update logic of our production `KalmanFilterPairs` class to ensure long-term stability during multi-year backtests.

Kalman Filter vs. Recursive Least Squares (RLS)

It is mathematically instructive to compare the Kalman Filter with another recursive estimator: **Recursive Least Squares (RLS)** with a forgetting factor $\lambda \in (0, 1]$. RLS solves the optimization problem:

$$\hat{\beta}_t = \arg \min_{\beta} \sum_{\tau=1}^t \lambda^{t-\tau} (Y_{\tau} - \beta X_{\tau})^2$$

The RLS recursive update equations are:

$$k_t = \frac{P_{t-1} X_t}{\lambda + X_t^T P_{t-1} X_t} \quad (20)$$

$$\hat{\beta}_t = \hat{\beta}_{t-1} + k_t (Y_t - \hat{\beta}_{t-1}^T X_t) \quad (21)$$

$$P_t = \frac{1}{\lambda} (1 - k_t X_t) P_{t-1} \quad (22)$$

Kalman-RLS Equivalence

The RLS filter is equivalent to a Kalman Filter with:

$$R_t = \lambda \quad (23)$$

$$Q_t = \left(\frac{1}{\lambda} - 1 \right) P_{t-1} \quad (24)$$

Under this mapping, the RLS gain k_t matches the Kalman Gain G_t , and the state updates align exactly.

While RLS uses a constant scalar discount λ , the Kalman Filter provides a more flexible state-space representation where process noise Q and measurement noise R can be tuned independently to match the physical characteristics of the spread dynamics.

Pipeline Position

Step	Module	Output
1	<code>TS_stationarity_test</code>	I(1) confirmation
2	<code>VAR_lag_select</code>	k^* lags
3	<code>johansen.py</code>	r^*, β
4	<code>fit_vecm.py</code>	α, β , ECT
4.5	<code>diagnostics.py</code>	Health gate
5a	<code>spread_builder.py</code>	S_t series, OLS β
5b	<code>half_life.py</code>	τ , Q heuristic
5c	<code>zscore.py</code>	Z_t , signals
6	<code>kalman_filter.py</code>	$\hat{\beta}_{t t}$, dynamic spread
6.1	<code>smoother.py</code>	$\hat{\beta}_{t T}$
6.2	<code>dynamic_beta.py</code>	Full pipeline
7	<code>vecm/kalman_forecast</code>	Forecasts

Interview Preparation

Key Interview Questions

1. **Why is the Kalman filter Bayesian?**

Prior $P(\beta_{t-1})$ + Likelihood $P(Y_t|\beta_t) \rightarrow$ Posterior $P(\beta_t|Y_t)$. The predict step is the prior; update is Bayes' rule applied.

2. **What does Kalman Gain represent?**

$G_t = P_{t|t-1} X_t / F_t$: the weight given to the new observation vs. prediction. High $G_t \rightarrow$ trust data; Low $G_t \rightarrow$ trust prediction.

3. **Why does P collapse fast and stay small?**

After the first update, $(1 - G_t X_t) P \ll P$. P only grows by Q per step. If Q is tiny, P stays near its converged steady-state value.

4. **How do you choose Q and R ?**

MLE: maximise $\sum_t \ell_t$ over (Q, R) . Heuristic: $Q \approx \sigma_{\epsilon}^2 / \tau$.

5. **What happens if $Q = 0$?**

$P_{t|t-1} = P_{t-1|t-1}$, $G_t \rightarrow 0$ as $t \rightarrow \infty$, and $\hat{\beta}_t \rightarrow \hat{\beta}_{OLS}$. The Kalman filter degenerates to a fixed estimate.